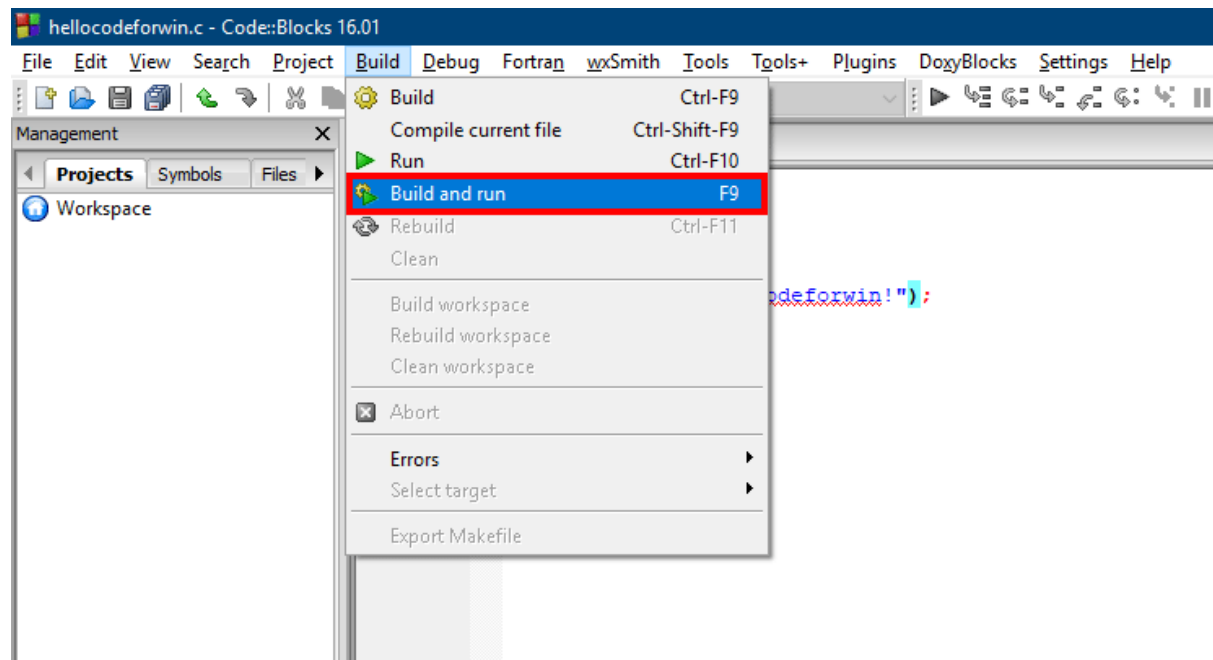


Welcome to CSE310

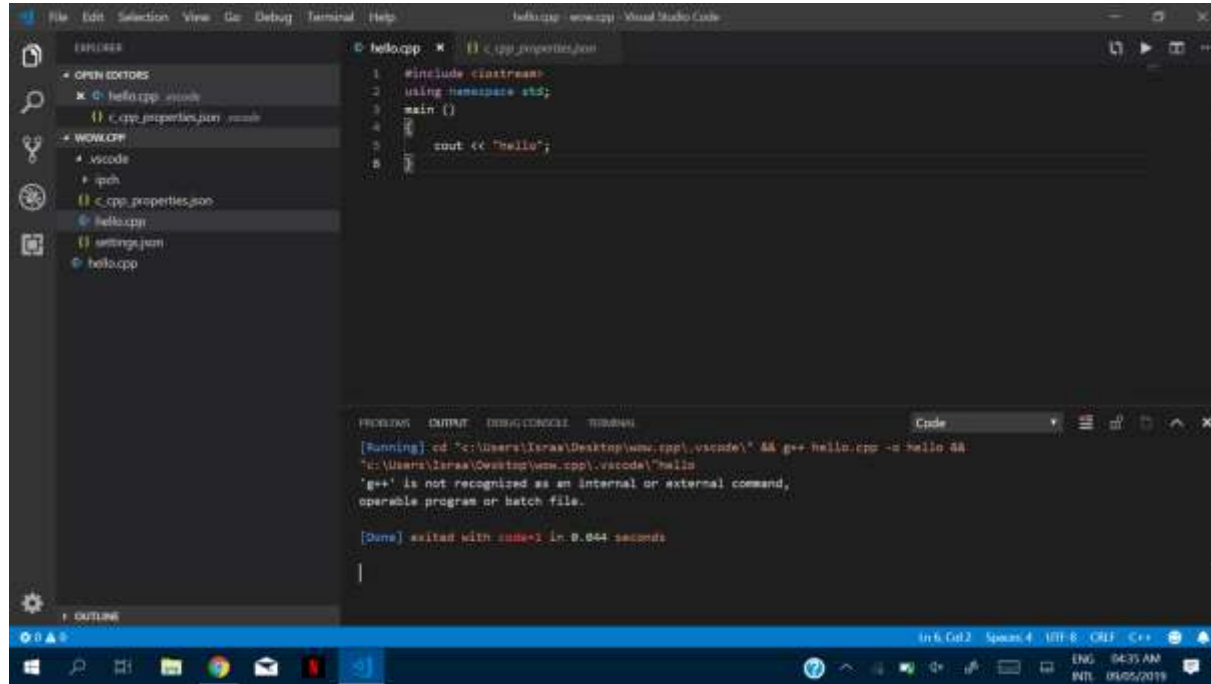
Compiler Sessional

Presenter: Md Irtiaz Kabir
Adjunct Lecturer, CSE, BUET

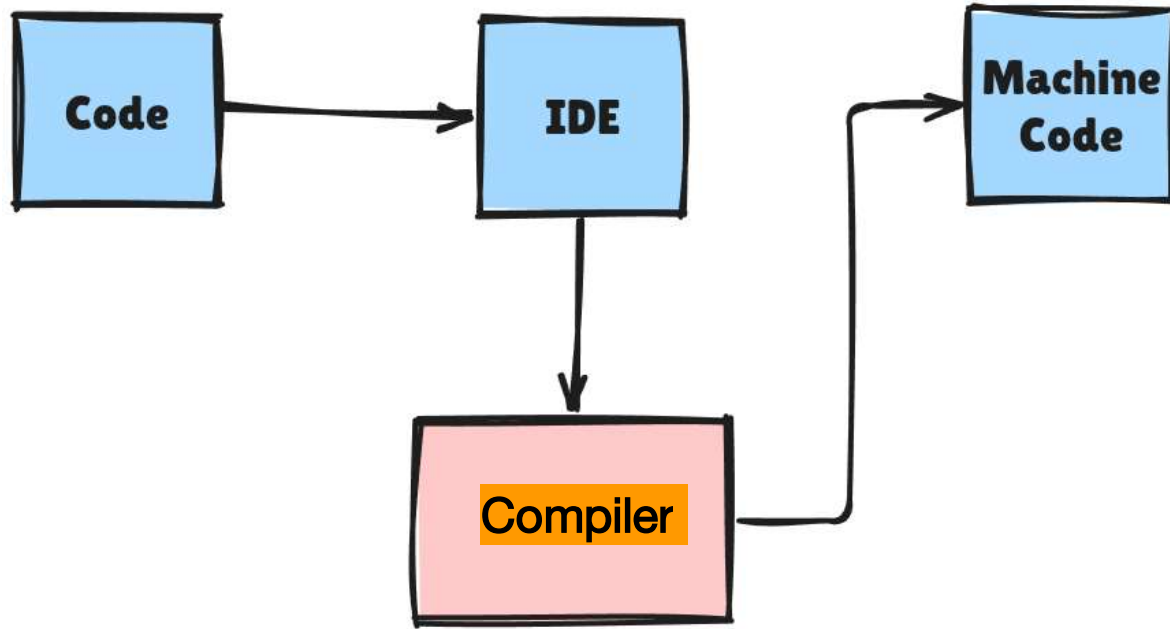
How we have compiled so far



How we have compiled so far



What happens under the hood



We will build a compiler!

- Learn how a compiler works
- Build a “toy” compiler
 - We focus on a subset of C
- Maybe have some fun in the process



Some important notes

- Write the entire thing **from scratch**
- **No plagiarism**
- **No LLM** generated codes



Back in my days...

A light blue rectangular box with a black border and a slight 3D effect, containing the word "Flex" in bold black text.

Flex

A light blue rectangular box with a black border and a slight 3D effect, containing the word "Bison" in bold black text.

Bison

A light blue rectangular box with a black border and a slight 3D effect, containing the word "Compiler" in bold black text.

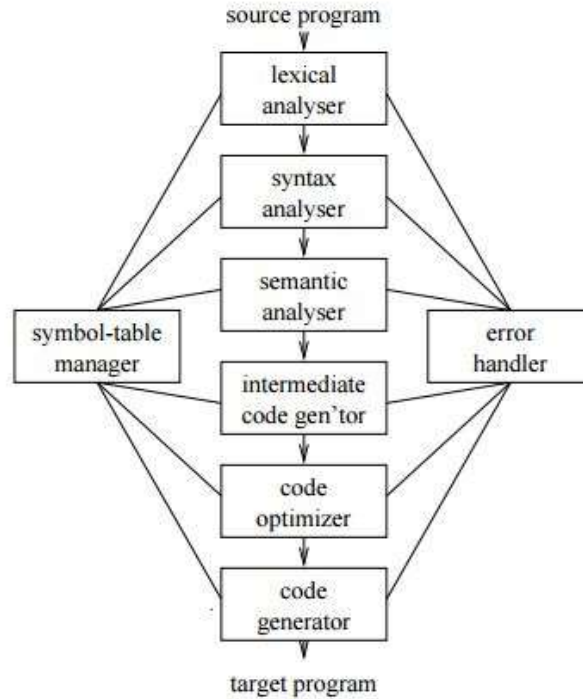
Compiler

- I wrote **my own flex + bison** (from scratch)
- Then my **own** programming language
- **Not a single LLM usage**
- In **ANSI C**
- During **PL**

```
poro n
jodi n, 10 er boro hoy taile
    likho 1
koro
noile jodi n, 5 er boro hoy taile
    likho 2
koro
ar noile
    likho 3
koro
```

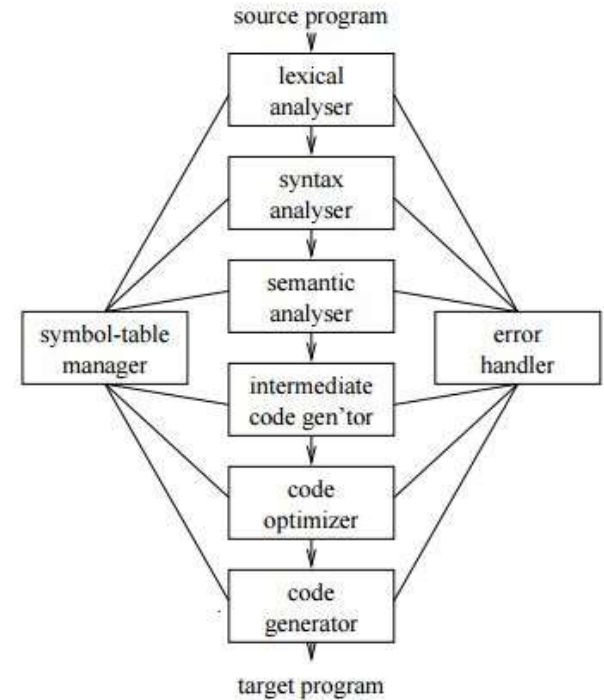
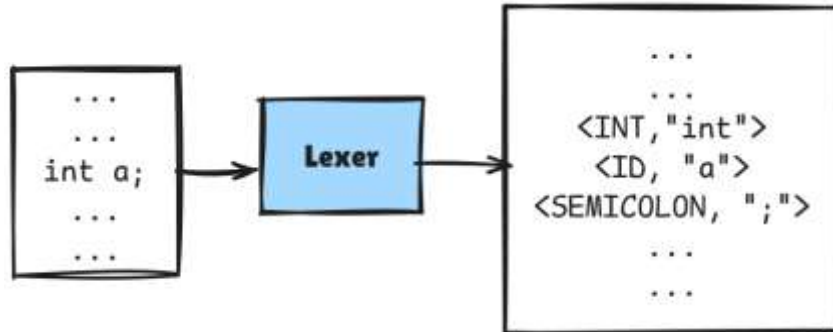
Jui Lang

Phases of a compiler



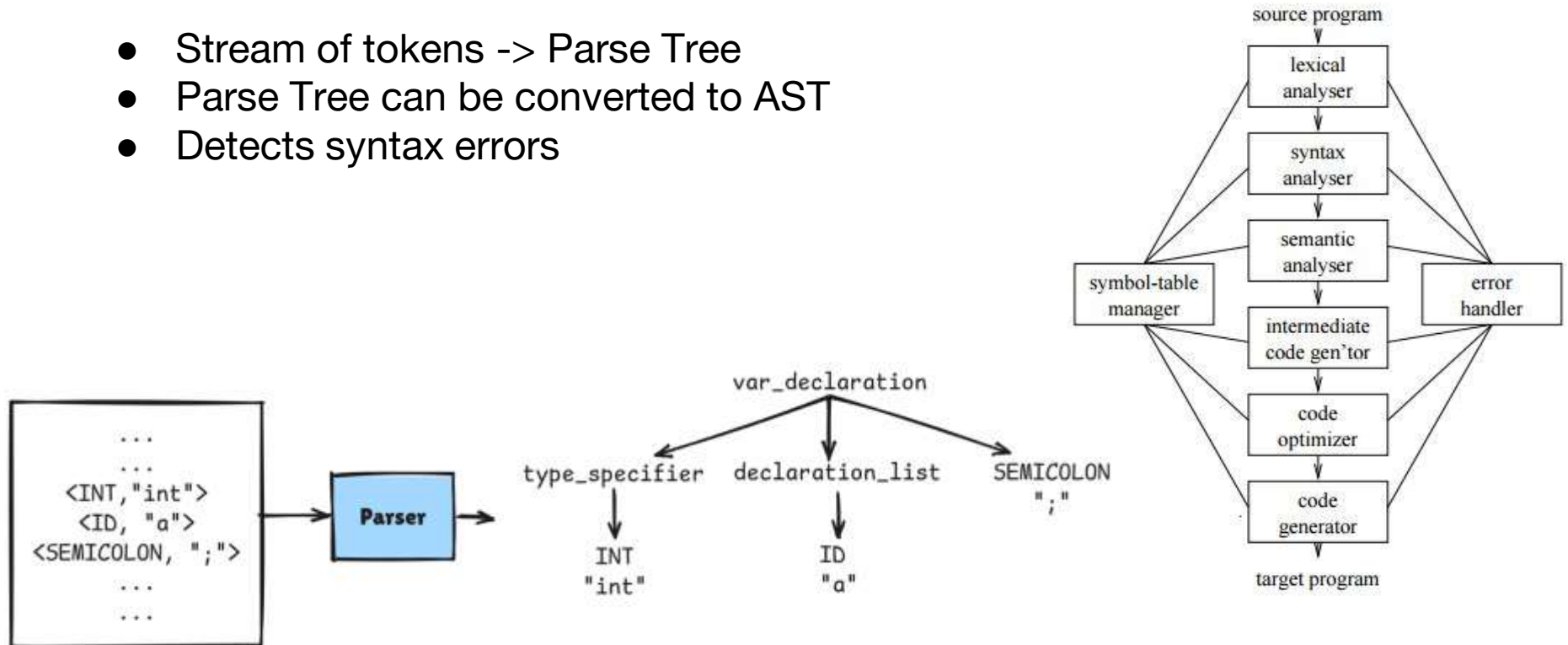
Lexical Analyzer

- Source program -> stream of tokens
- Passes tokens to syntax analyzer
- Detects lexical errors
 - ill formed numbers
 - Improper variable declaration
 - Unfinished strings/comments



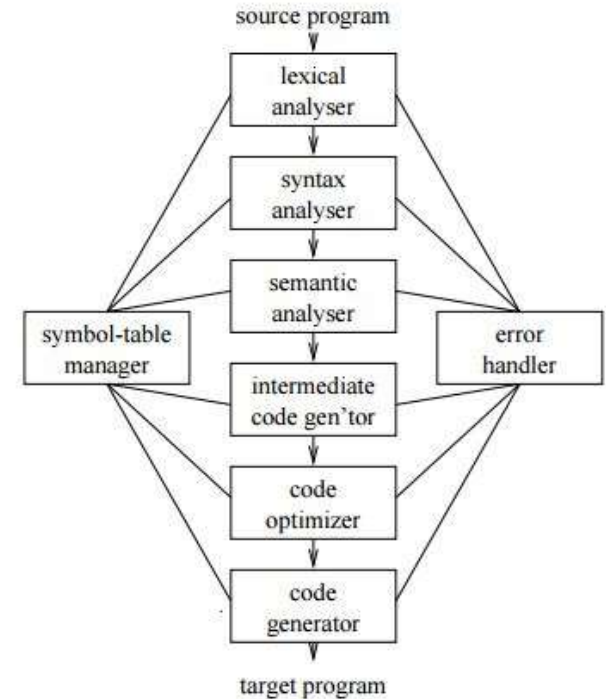
Syntax Analyzer

- Stream of tokens -> Parse Tree
- Parse Tree can be converted to AST
- Detects syntax errors

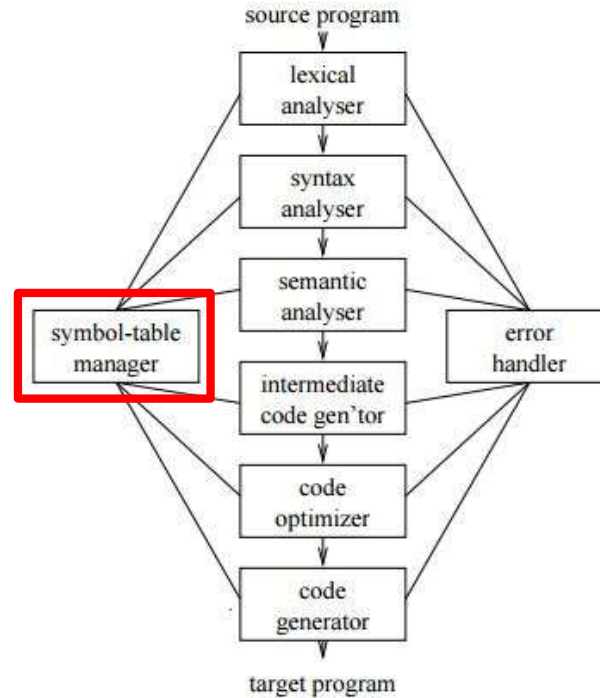


Semantic Analyzer

- Check semantic consistency
- Can add nodes in the AST
 - Implicit type conversions
- Detects semantic errors
 - Type checking
 - Variable declared as void
 - Undeclared variable
 - Error in number of or type of arguments in function call



We first need a symbol table



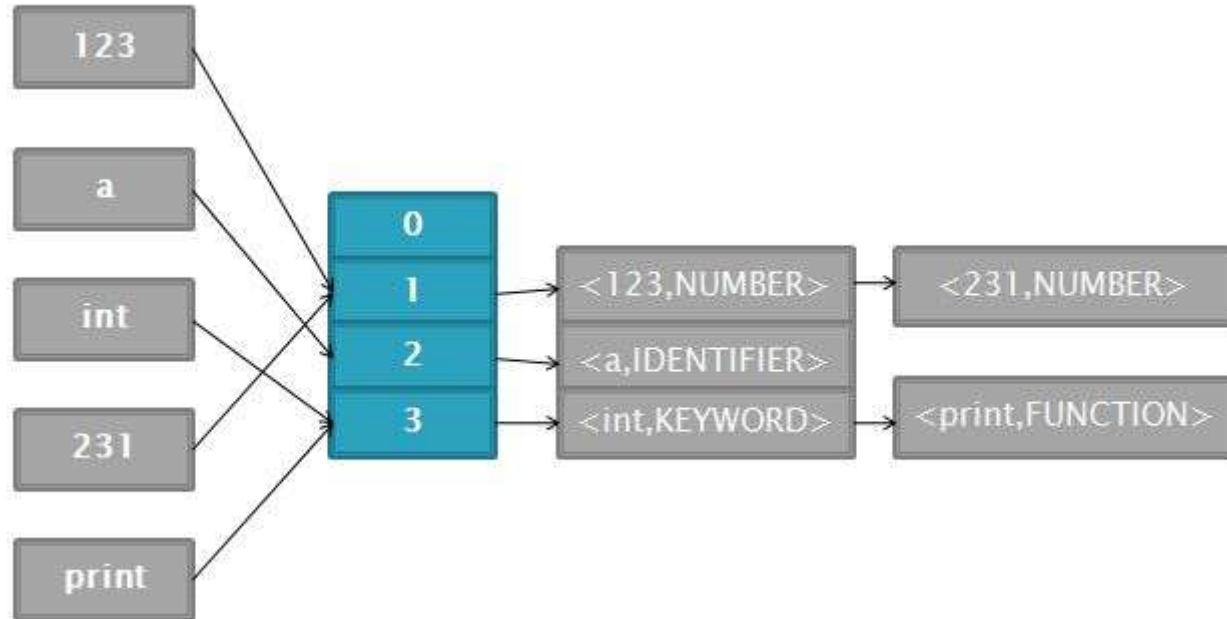
Symbol table

- A table to store symbol information
 - Symbol name
 - Symbol type
 - Scope etc
- Almost all phases refer to the symbol table for type checking

Basically a hash table

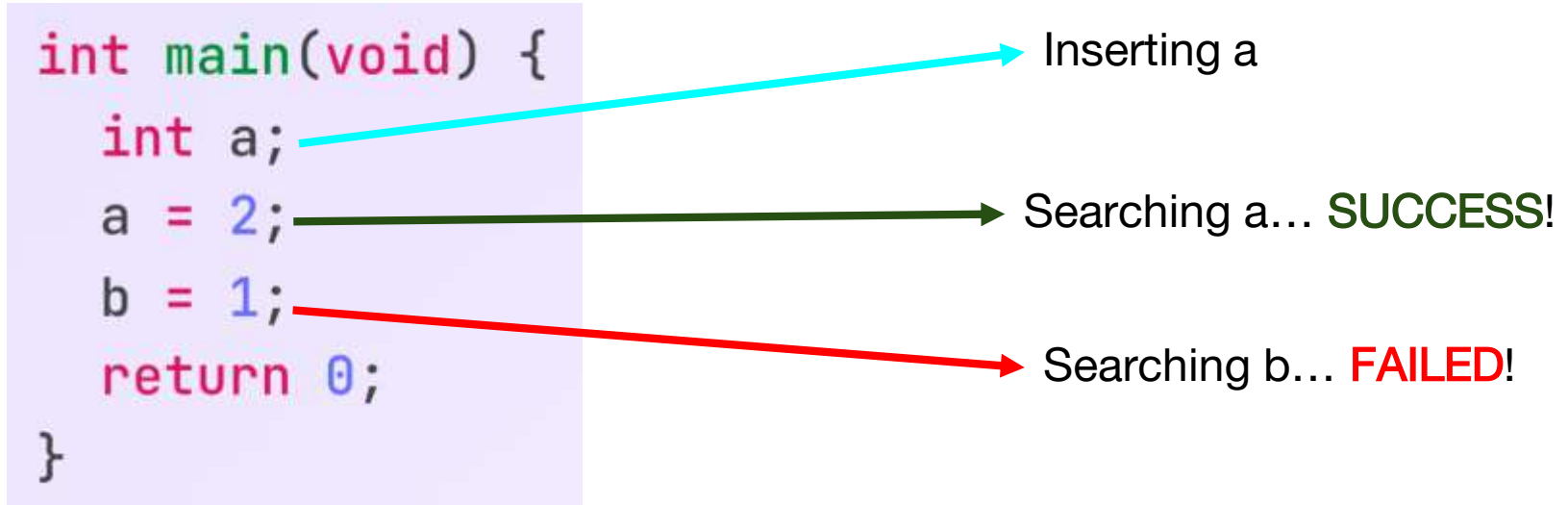
- Symbol table is just like a hash table
- Each entry in the hash table is **mainly** a two tuple
 - <Symbol Name, Symbol Type>
- Symbol Name is used as the key for hashing
- Need to resolve hash collisions
 - We use **separate chaining**

Basically a hash table



How does a symbol table help?

- Detect undeclared variables



How does a symbol table help?

- Type checking

```
int main(void) {  
    int a;  
    a = 2.5;  
    return 0;  
}
```

Type mismatch... Implicit cast to int

How does a symbol table help?

- Scope Management
 - Need to allow duplicate entries
 - Need to delete some entries when a block ends

```
int main(void) {  
    int a;  
    {  
        int a,b;  
    }  
    b=2;  
    return 0;  
}
```

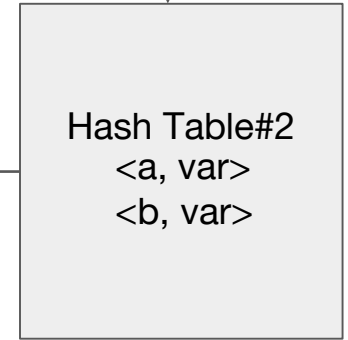
List of Hash Tables

```
→ int main(void) {  
→   int a;  
→   {  
→     int a,b;  
→   }  
→   b=2;  
→   return 0;  
→ }
```

Current Pointer



Current Pointer



Parent pointer

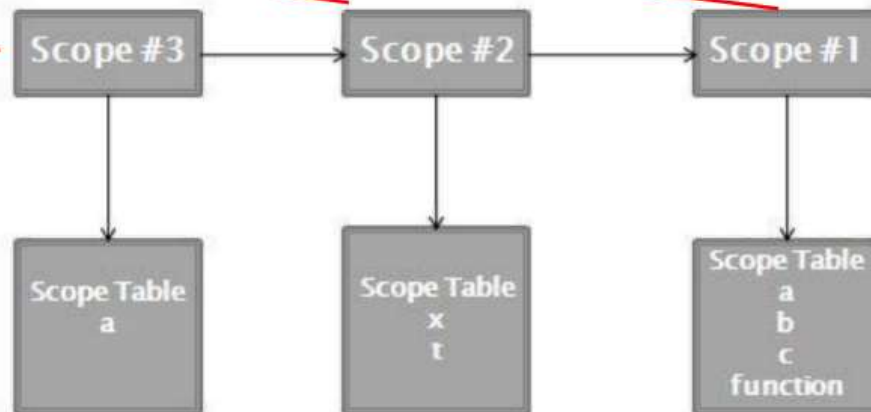


Stack of Hash Tables

- Instead of a list and a current pointer, we can maintain a stack
- Current scope is always the **top of the stack**
- When accessing,
 - Search the topmost stack first
 - If failed, search its parent and so on

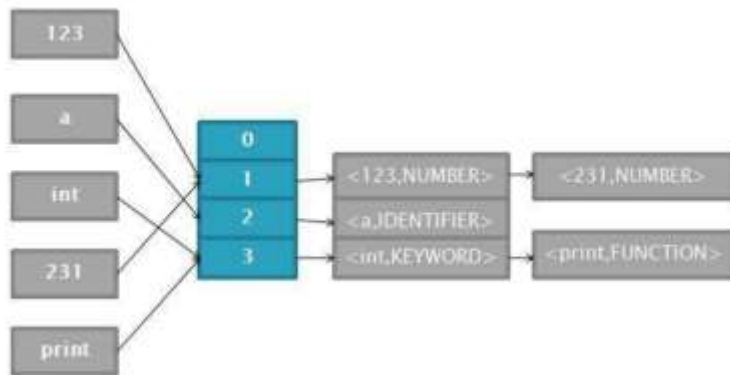
Stack of Hash Tables

```
1. int a, b, c;
2. int func(int x) {
3.     int t = 0;
4.     if(x == 1) {
5.         int a = 0;
6.         t = 1;
7.     }
8.     return t;
9. }
10. int main() {
11.     int x = 2;
12.     func(x);
13.     return 0;
14. }
```



Three Classes: **SymbolInfo**

- Each entry of symbol table is an instance of SymbolInfo
 - Remember two tuples!!!
- Three member vars
 - Name
 - Type
 - Pointer to another SymbolInfo class (needed for chaining in hash table)



Three Classes: ScopeTable

- A regular hash table
- Represents each scope in the code
- Implements four operations
 - Insert
 - Lookup
 - Delete
 - Print

Three Classes: **SymbolTable**

- Maintains a list of Scope Tables (or a stack)
- Implements six operations
 - Enter Scope
 - Exit Scope
 - Insert
 - Delete
 - Print All Scope Tables
 - Print Current Scope Tables

Cautions

- There will be an **online**
 - Plagiarism will make you suffer later
- Symbol table is used in all the later offline assignments
 - Any bug might snowball into a complex issue later
- No memory leaks tolerated
 - Use **-fsanitize=address** flag for auto leak detection in runtime

Acknowledgement

- Ajmain Yasar Ahmed Sahil
- Nafis Tahmid

Thank You!
Questions?